Lesson 5 — Introduction to Databases and SQL

Node.js/Express

Game Plan

- Warm-Up
- Why Databases?
- SQL Basics
- Associations & Transactions
- Joins
- Using `pg` in Node
- Parameterized Queries & Security
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. Have you worked with a database before? SQL or otherwise?
2. What happens to the data in your app right now when you restart the server?

The Problem with Memory Storage

Right now, you store data like this:

```
global.users = [];
global.tasks = [];
```

Problems:

- Data disappears when the server restarts
- Can't scale to multiple server instances
- Limited to available RAM

A **database** fixes all of this.

Why Relational Databases?

Relational databases store data in **tables** (like spreadsheets).

Each table has:

- **Columns** — the fields (id, name, email)
- **Rows** — each record
- **Constraints** — rules about the data (NOT NULL, UNIQUE)

And tables can **relate to each other** via foreign keys.

SQL: The Language

SQL (Structured Query Language) works with all relational databases.

The key verbs:

- `SELECT` — read data
- `INSERT` — add data
- `UPDATE` — change data
- `DELETE` — remove data
- `CREATE TABLE` / `DROP TABLE` — manage structure

SELECT

```sql
-- Get all users
SELECT * FROM users;

-- Get specific columns
SELECT name, email FROM users WHERE email = 'alex@test.com';

-- Sort and limit
SELECT * FROM tasks ORDER BY created_at DESC LIMIT 10;
```

The `WHERE` clause filters rows.

Without it on a `DELETE`, you delete **everything**.

INSERT & UPDATE

```sql
-- Add a new user
INSERT INTO users (name, email) VALUES ('Alex', 'alex@test.com');

-- Update a task
UPDATE tasks SET is_completed = true WHERE id = 5;

-- Return what was just created
INSERT INTO tasks (title, user_id)
VALUES ('Buy groceries', 1)
RETURNING id, title;
```

`RETURNING` is PostgreSQL-specific and very handy.

Primary Keys & Foreign Keys

Primary key — uniquely identifies a row:

```
id SERIAL PRIMARY KEY  -- auto-incrementing integer
```

Foreign key — points to a row in another table:

```
user_id INTEGER NOT NULL REFERENCES users(id)
```

This enforces the relationship: a task must belong to a real user.

Associations

One-to-many: One user has many tasks.

```
users             tasks
------            ------
id: 1     ←       user_id: 1
id: 2     ←       user_id: 2
                  user_id: 2
```

Many-to-many: One order can have many products; one product can be in many orders.

Solution: a **join table** (e.g., `line_items`).

Transactions

A transaction ensures multiple operations **all succeed or all fail**:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

If the second UPDATE fails, the first is **rolled back**.

This is how databases stay consistent.

JOINS

Combine data from multiple tables:

```
SELECT tasks.title, users.name
FROM tasks
JOIN users ON tasks.user_id = users.id
WHERE users.email = 'alex@test.com';
```

`JOIN` matches rows from each table using the `ON` condition.

Without a match in an INNER JOIN, the row is excluded.

Predict This (2 min)

What does this SQL do?

```sql
DELETE FROM tasks;
```

And this one?

```sql
DELETE FROM tasks WHERE user_id = 5;
```

PostgreSQL in Node: the `pg` Package

```
const { Pool } = require("pg");

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});
```

Use a **pool** — it maintains multiple reusable connections.

All queries go through `pool.query()`.

Running a Query

```
const result = await pool.query(
  "SELECT * FROM users WHERE email = $1",
  [email]
);

const users = result.rows;  // array of row objects
```

Notice `$1` — that's a **parameter placeholder**.

The second argument is the array of values to substitute.

Why Parameterized Queries?

Never concatenate user input into SQL:

```
// DANGEROUS ❌
const query = `SELECT * FROM users WHERE email = '${email}'`;
```

An attacker could enter:

```
' OR 1=1; DROP TABLE users; --
```

Always use parameterized queries:

```
// SAFE ✅
pool.query("SELECT * FROM users WHERE email = $1", [email]);
```

We Do — Read a Schema

Given this schema:

```
CREATE TABLE tasks (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  is_completed BOOLEAN DEFAULT FALSE,
  user_id INTEGER NOT NULL REFERENCES users(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

What constraints are there? What would fail to insert?

We Do — Write a Query

Write a SQL query to:

Fetch all tasks for user with id `3`, ordered by `created_at` descending.

```
-- Your answer:

```

You Do (5 min)

Using the `pg` package pattern, write a function that:

1. Accepts a `userId`
2. Returns all tasks for that user from the database

```
async function getTasksByUser(userId) {
  // your code here
}
```

Assignment Preview

Assignment 5 has two parts:

5a: Practice SQL in a command line tool — run queries against a pre-built database (customers, orders, products).

5b: Convert your app from `global.tasks` to real PostgreSQL:

- Set up the `pg` pool
- Write SQL queries in your task and user controllers
- Handle database errors

Assignment: Schema

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(30) NOT NULL,
  hashed_password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Note: `global.user_id` still exists temporarily — you'll remove it in Lesson 8.

Wrap-Up

In chat:

1. What SQL verb is used to read data?
2. Why are parameterized queries safer than string concatenation?
3. What is a foreign key and what does it enforce?

Confidence Check

On a scale of 1–5:

How comfortable are you with writing SQL queries?

Resources

- https://sqlbolt.com/ (excellent interactive SQL exercises)
- https://node-postgres.com/ (pg documentation)
- https://www.postgresql.org/docs/
- Ask questions in Slack

Closing

This week:

SQL, PostgreSQL, and connecting your Node app to a real database.

Next week:

Replace raw SQL with Prisma — an Object-Relational Mapper that makes database work feel more like JavaScript.

See you then!